Assignment 11: Implementing a Persistence Repository Layer

***NOTE: Please Create backlog/issues for all these activities you are doing ***

Objective:
Design and implement a **repository layer** to persist your domain model objects. The implementation should:

1. **Abstract storage details** behind an interface (allowing easy swapping of storage backends).
2. Support **CRUD operations** (Create, Read, Update, Delete) for domain entities.
3. Use **Dependency Injection (DI)** or **Factory Pattern** to switch between storage implementations.

Scenario

Your system now has well-defined domain classes (from Assignment 10). To make the application **persistent**, you must:

- Build a **repository layer** to store/retrieve objects.
- Start with an **in-memory HashMap** for simplicity, but design it to be **easily replaceable** with:
  - Filesystem storage (JSON/XML files)
  - SQL/NoSQL databases (MySQL, MongoDB)
  - External REST APIs

Tasks

1. Repository Interface Design (30 Marks)

Task:

- Define a **generic repository interface** with standard CRUD operations:

```
public interface Repository<T, ID> {
    void save(T entity);          // Create/Update
    Optional<T> findById(ID id);  // Read
    List<T> findAll();            // Read All
    void delete(ID id);           // Delete
}
```

- Create **entity-specific interfaces** (e.g., `BookRepository extends Repository<Book, String>`).

Deliverables:

- A `/repositories` **directory** with interface definitions.
- **Justification** in the README (e.g., *"Used generics to avoid duplication across entity repositories"*).

---

2. In-Memory Implementation (30 Marks)

Task:

- Implement the repository using an **in-memory** `HashMap` / `Dictionary`.
- Example (Python):

```python
class InMemoryBookRepository(BookRepository):
    def __init__(self):
        self._storage = {}  # HashMap storage

    def save(self, book: Book) -> None:
        self._storage[book.id] = book

    def find_by_id(self, id: str) -> Optional[Book]:
        return self._storage.get(id)
```

Deliverables:

- A `/repositories/inmemory` **directory** with HashMap-based implementations.
- **Test cases** verifying CRUD operations work correctly.

---

3. Storage-Abstraction Mechanism (20 Marks)

Task:

- Ensure the **repository layer is decoupled** from storage specifics.
- Use one of these patterns:
  - **Dependency Injection (DI)**: Inject the repository implementation into services.
  - **Factory Pattern**: A `RepositoryFactory` returns the desired storage backend.

Example (Factory in Java):

```
public class RepositoryFactory {
    public static BookRepository getBookRepository(String storageType) {
        switch (storageType) {
            case "MEMORY": return new InMemoryBookRepository();
            case "DATABASE": return new DatabaseBookRepository(); // Future
implementation
            default: throw new IllegalArgumentException("Invalid storage
type");
        }
    }
}
```

Deliverables:

- A `/factories` or `/di` **directory** with your abstraction mechanism.
- **README update** explaining your choice (DI vs. Factory).

---

4. Future-Proofing (20 Marks)

Task:

- Design your system to **easily add new storage backends** later.
- Example: A `FileSystemBookRepository` could serialize objects to JSON:

```
class FileSystemBookRepository(BookRepository):
    def __init__(self, file_path: str):
        self._file_path = file_path

    def save(self, book: Book) -> None:
        books = self._load_all()
        books[book.id] = book
        with open(self._file_path, 'w') as f:
            json.dump(books, f)
```

Deliverables:

- A **stub implementation** (no full code needed) for one future storage option (e.g., `DatabaseBookRepository`).
- **Updated class diagram** showing repository interfaces and implementations.

Deliverables

1. **Repository Interfaces**:
   - Generic and entity-specific interfaces (`/repositories`).
2. **In-Memory Implementation**:
   - HashMap-based repositories (`/repositories/inmemory`).
3. **Abstraction Mechanism**:
   - DI or Factory setup (`/factories` or `/di`).
4. **Future-Proofing Docs**:
   - Stub for another storage backend.
   - Updated class diagram.
5. **Tests**:
   - Unit tests for in-memory repositories (`/tests`).

Submission Guidelines

- **IMPORTANT** : Please check your URL that you submit to BB. Don't do it mindlessly
- **Format**: Push to your existing GitHub repository and post a VALID LINK to Blackboard.
- **Grading**:
  - **30%**: Repository interface design (generics, CRUD completeness).
  - **30%**: Working in-memory implementation.
  - **20%**: Proper abstraction (DI/Factory) for storage swapping.
  - **20%**: Future-proofing and documentation.

Why This Matters

- **Separation of Concerns**: Business logic stays cleanly separated from storage details.
- **Scalability**: Switching to a real database later becomes trivial.
- **Testability**: In-memory repos enable fast unit tests without external dependencies.

Need Help?

- **Repository Pattern**: Microsoft Docs
- **Dependency Injection**: Martin Fowler's Article
